

Customer Churn Prediction in Retail E-commerce

Pipeline complet ML : Preprocessing · PCA · Clustering · Classification · Régression · Flask

Yassine Meftah

Atelier Machine Learning | Année 2025–2026

Contexte & Problématique

Contexte

- Les entreprises e-commerce subissent un taux de churn élevé
- Fidéliser un client coûte 5× moins cher qu'en acquérir un nouveau
- Les données comportementales et transactionnelles offrent des signaux prédictifs
- L'enjeu est d'agir avant que le client parte

Problématique

Peut-on prédire quels clients risquent de quitter la plateforme à partir de leurs données comportementales et transactionnelles ?

Objectif principal :
Construire un pipeline ML capable de prédire le churn, d'explorer les profils clients et de proposer une solution exploitable.

Objectifs du Projet

Prédiction du Churn

Modèles de classification supervisée pour identifier les clients à risque

Analyse des Variables

Identifier les features les plus prédictives du comportement client

Réduction PCA

Réduire la dimensionnalité pour visualisation et clustering

Segmentation KMeans

Explorer les profils clients de manière non supervisée

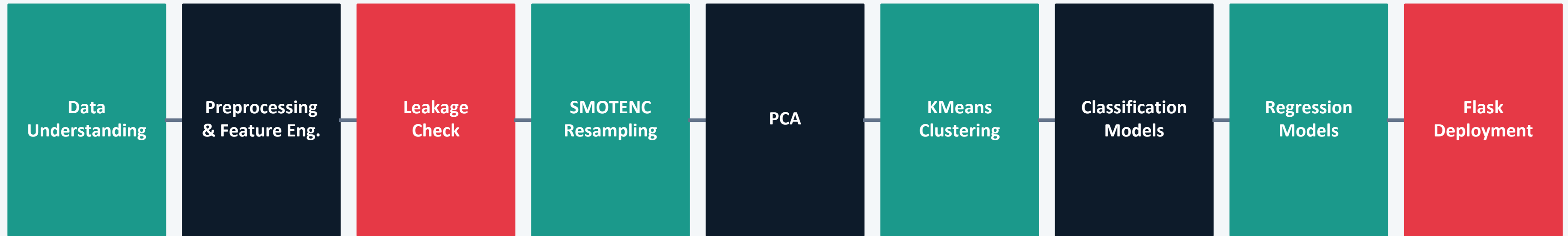
Régression MonetaryTotal

Prédire la valeur monétaire totale d'un client

Déploiement Flask

Interface web pour tester le modèle sur de nouveaux profils

Pipeline Global du Projet



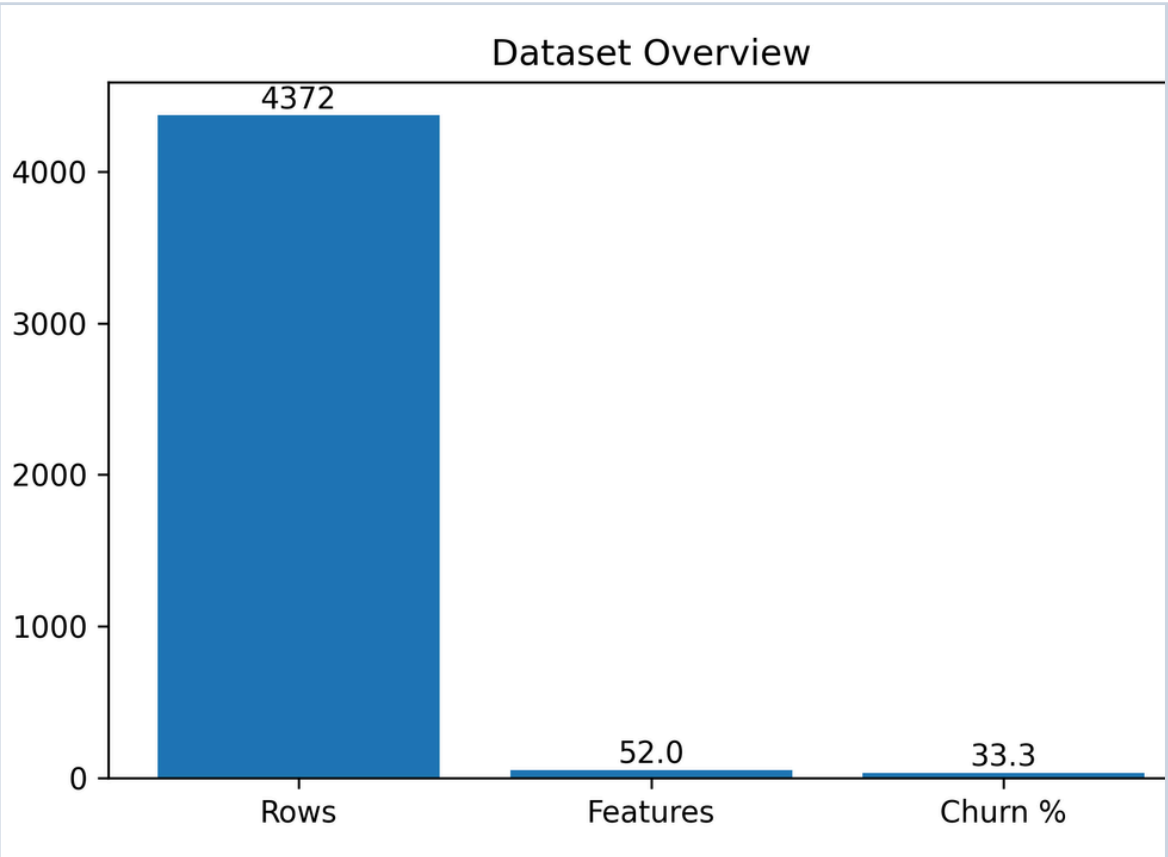
Approche Méthodologique

- Pipeline ML complet de bout en bout
- ✓ Méthodologie orientée production
- ✓ Gestion rigoureuse du data leakage
- ✓ Séparation claire train / test
- ✓ Approche reproductible et robuste

Présentation du Dataset

Description générale

- Dataset e-commerce retail
- Variable cible : Churn (binaire)
- Variables transactionnelles, temporelles, démographiques et comportementales
- Plusieurs problèmes de qualité à corriger



Transactionnelles

Frequency, MonetaryTotal, UniqueProducts, ReturnRatio, AvgQuantityPerTransaction

Temporelles

CustomerTenureDays, FirstPurchaseDaysAgo, PreferredHour, PreferredMonth, AvgDaysBetweenPurchases

Démographiques

Age, Gender, Country, Region, AccountStatus

Comportementales

SupportTicketsCount, SatisfactionScore, WeekendPurchaseRatio, ZeroPriceCount

Problèmes Rencontrés au Départ

? Valeurs manquantes

Présence de valeurs manquantes, notamment sur Age
→ Nécessité d'une imputation avancée (KNNImputer)

Dates à parser

Variable RegistrationDate non structurée
→ Extraction nécessaire (année, mois, jour, weekday)

Outliers

Valeurs aberrantes détectées dans : SupportTicketsCount et SatisfactionScore
→ Traitement par IQR clipping

Variables redondantes

Plusieurs variables fortement corrélées ou dupliquées
(ex : variables RFM, indicateurs similaires)
→ Risque de multicolinéarité

Déséquilibre de classes

Classe Churn=1 sous-représentée
→ Risque de biais du modèle vers la classe majoritaire
→ Nécessité de rééquilibrage (SMOTENC)

Data Leakage !

Performances initiales anormalement élevées
Exemples : AccountStatus, LoyaltyLevel, SpendingCategory
→ Suppression obligatoire pour éviter un modèle irréaliste

Préparation des Données — Étape par Étape

1

Suppression colonnes inutiles

NewsletterSubscribed, CustomerID, ChurnRiskCategory, Recency et dérivées → leakage

2

Parsing des dates

RegistrationDate → RegYear, RegMonth, RegDay, RegWeekday

3

Feature engineering IP

LastLoginIP → IsPrivateIP + IpFirstOctet

4

Outliers (IQR)

Clipping sur train uniquement, bornes sauvegardées et réappliquées sur test

5

Imputation

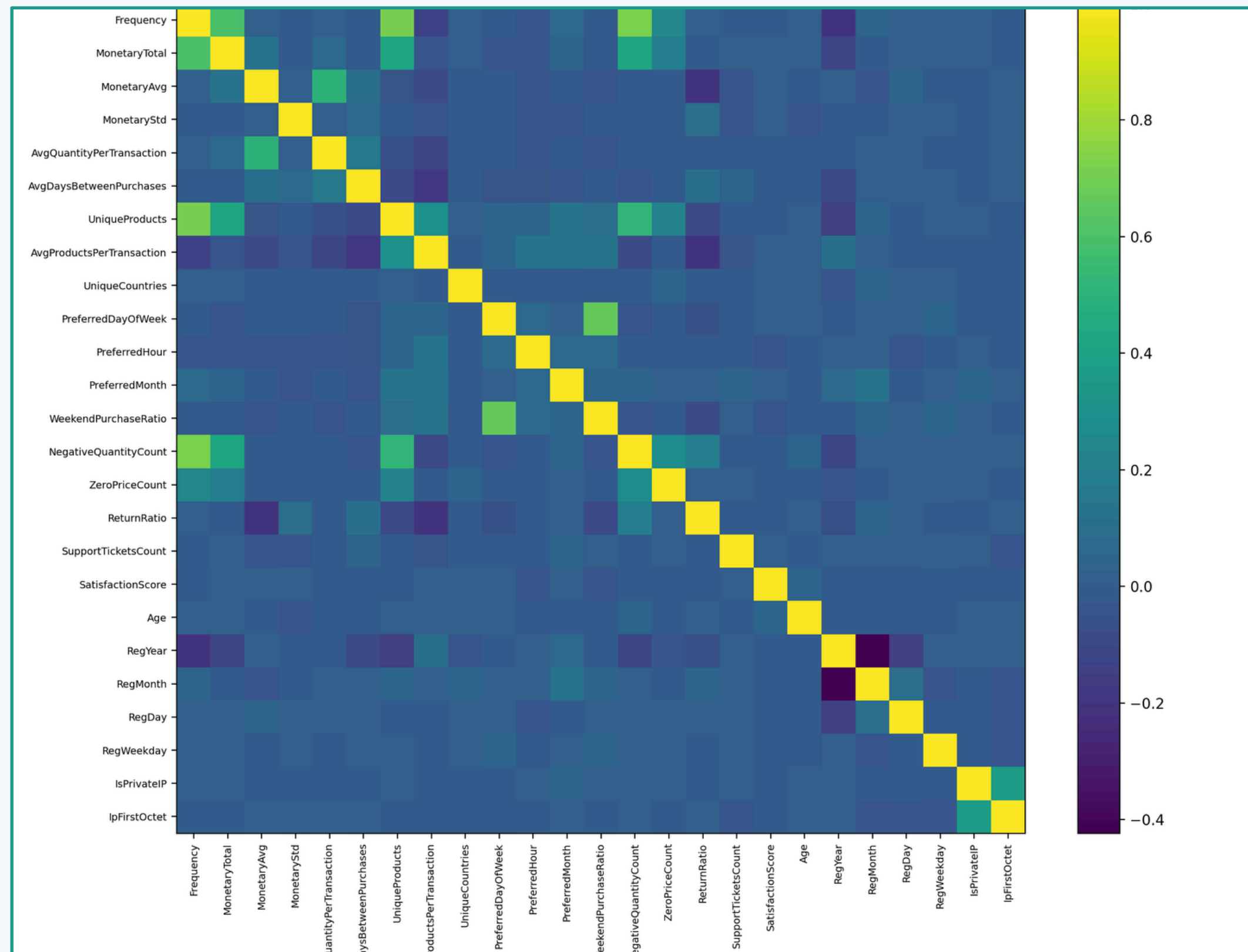
Médiane (numériques) · Mode (catégorielles) · KNNImputer (Age)

6

Encodage & Normalisation

OneHotEncoder (catégorielles) + StandardScaler (numériques)

Corrélation & Multicolinéarité



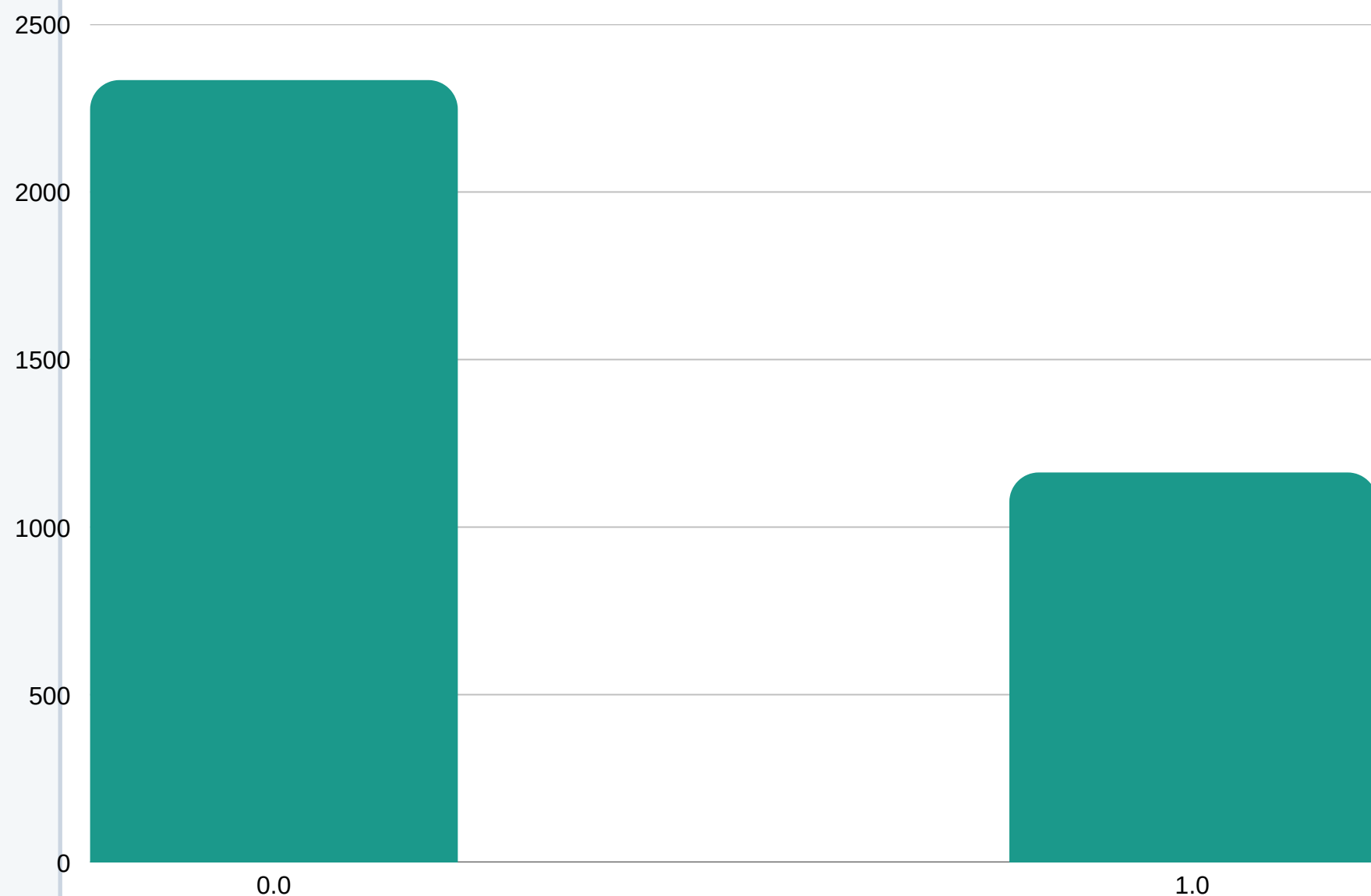
Analyse

- Plusieurs variables fortement corrélées identifiées
- Suppression automatique : seuil > 0.8
- 7 colonnes redondantes supprimées manuellement
- VIF calculé pour détecter la multicolinéarité sévère
- Variables finales : plus propres et plus pertinentes

Gestion du Déséquilibre : SMOTENC

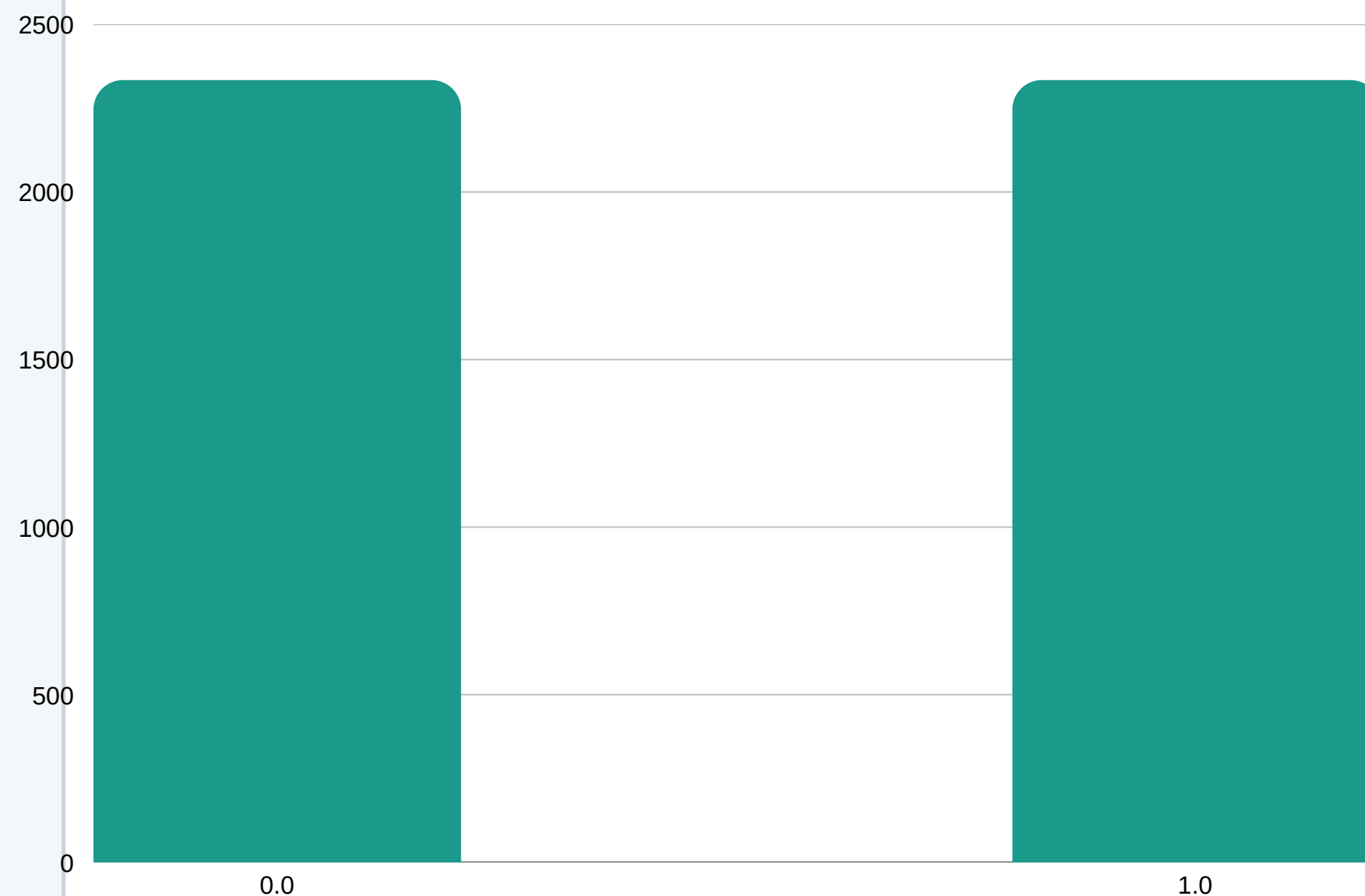
Avant SMOTENC

⚠ Classes déséquilibrées



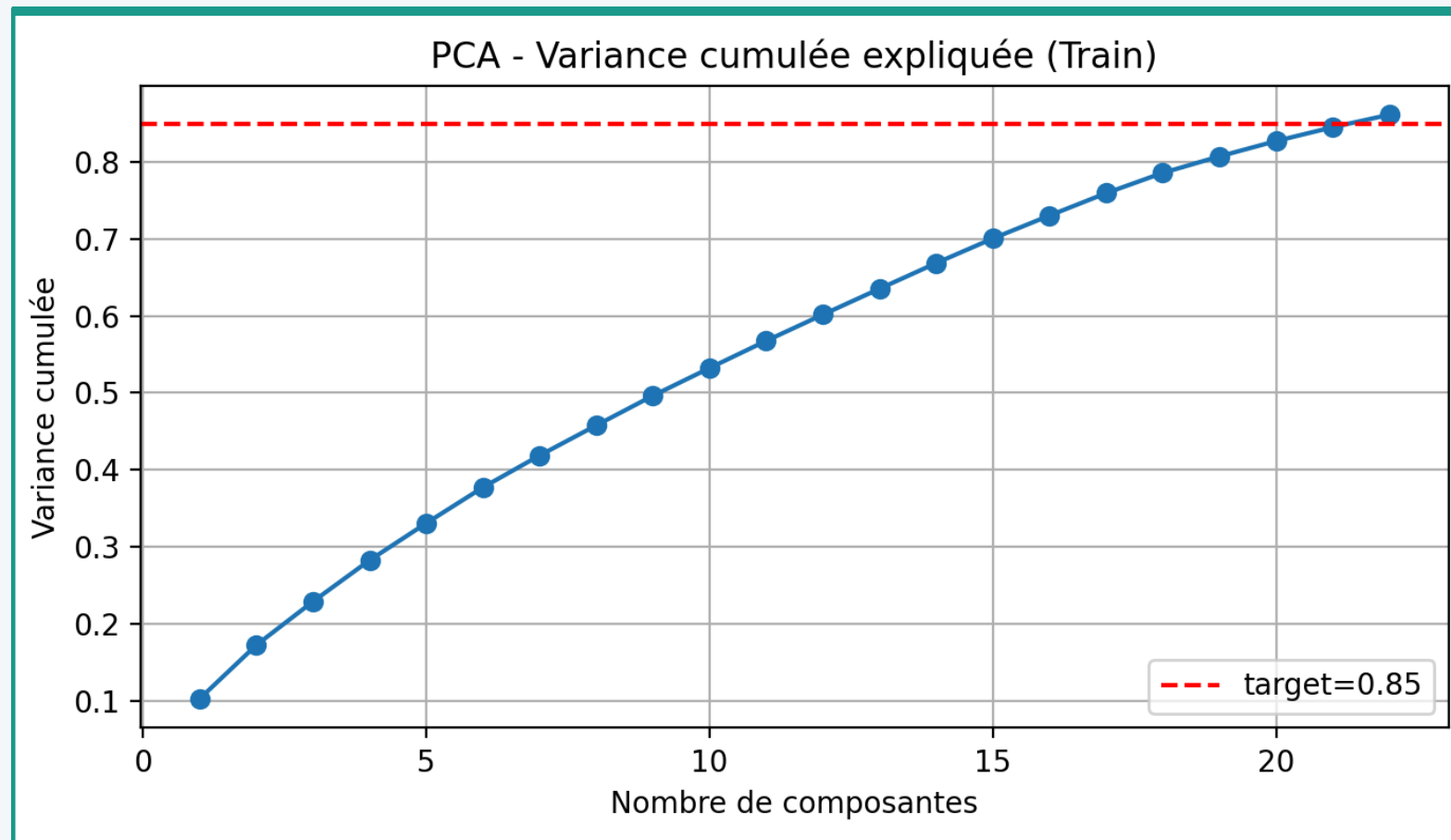
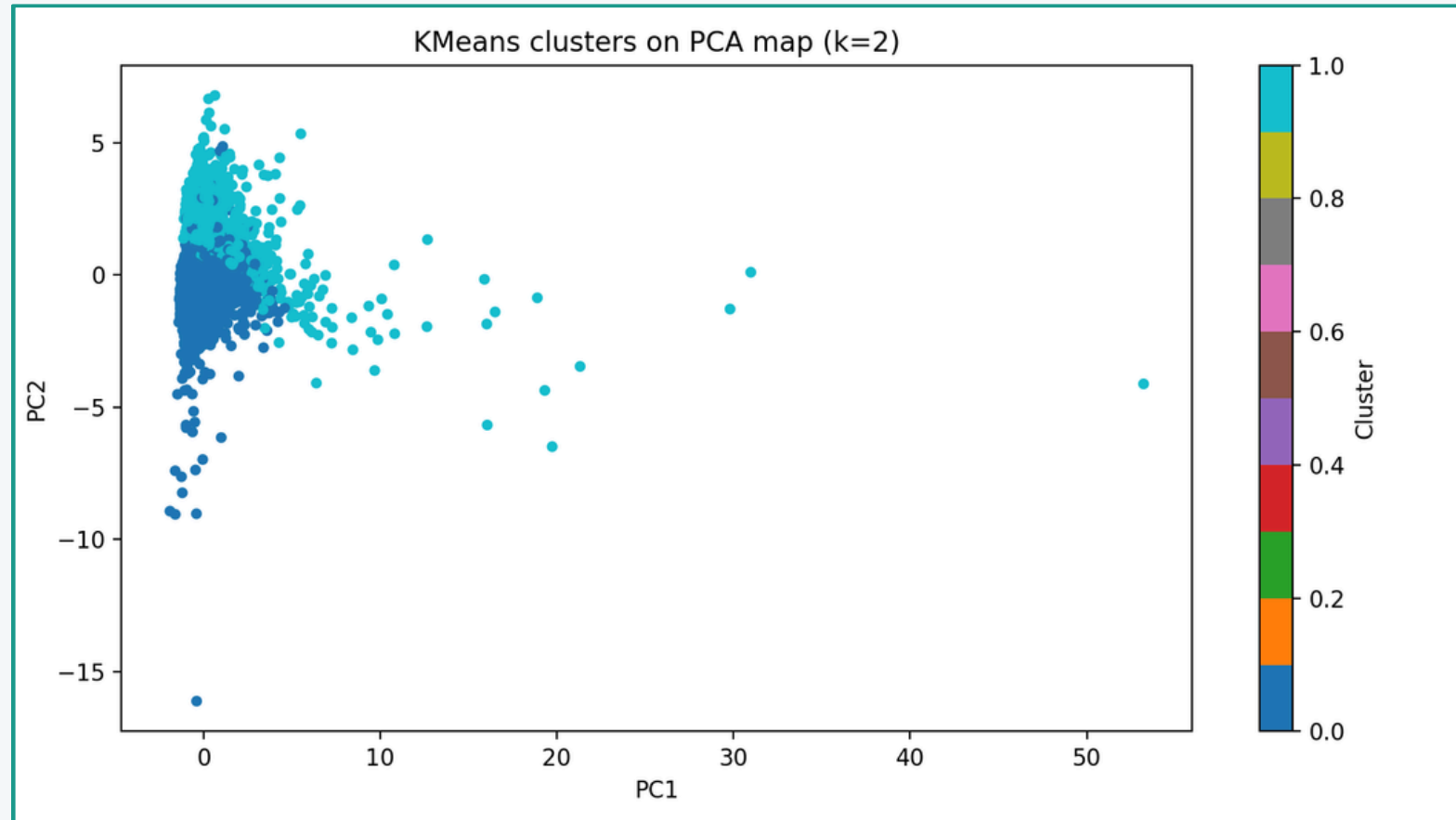
Après SMOTENC

✅ Classes équilibrées



SMOTENC appliqué uniquement sur le train | 3 497 → 4 668 observations | Données mixtes (numériques + catégorielles)

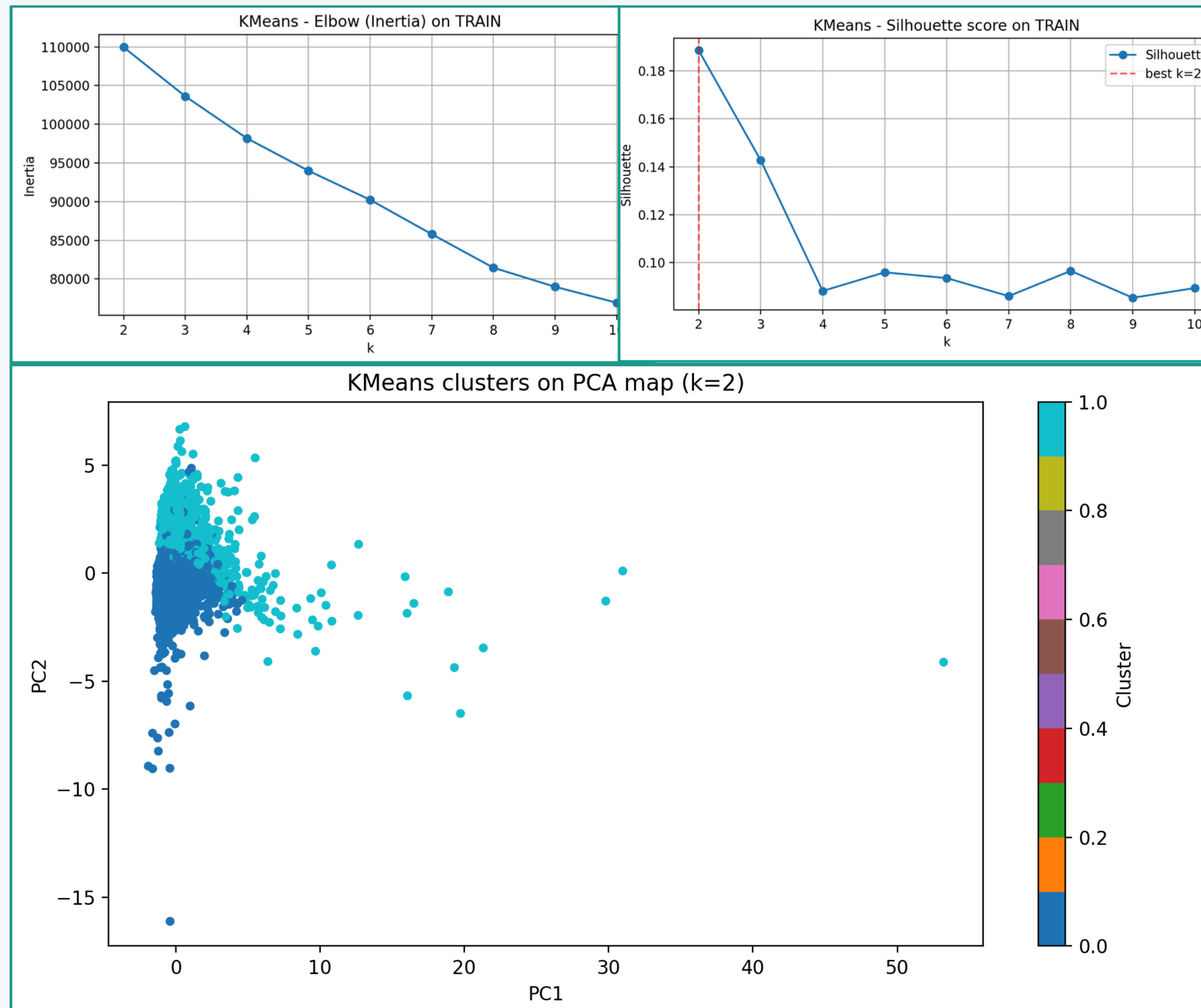
Réduction de Dimension : PCA



Résultats PCA

- Seuil de variance cumulée fixé à 85%
- Nombre de composantes retenues : 22
- Variance cumulée expliquée : $\approx 86\%$
- Réduction significative de la dimensionnalité
- Projection en 2D (PC1, PC2) pour visualisation
- PCA utilisé pour :
 - visualisation des données
 - préparation au clustering (KMeans)
- Permet de réduire le bruit tout en conservant l'information essentielle

Segmentation Clients : KMeans



Résultats

- k testé : 2 → 10 (Elbow + Silhouette)
- k optimal : 2
- Silhouette faible (~ 0.18)
→ séparation limitée
- Cluster 0 : clients majoritaires
- Cluster 1 : profils atypiques
- Segmentation exploratoire
→ complément de la classification
→ clusters peu distincts

Modélisation : Classification du Churn

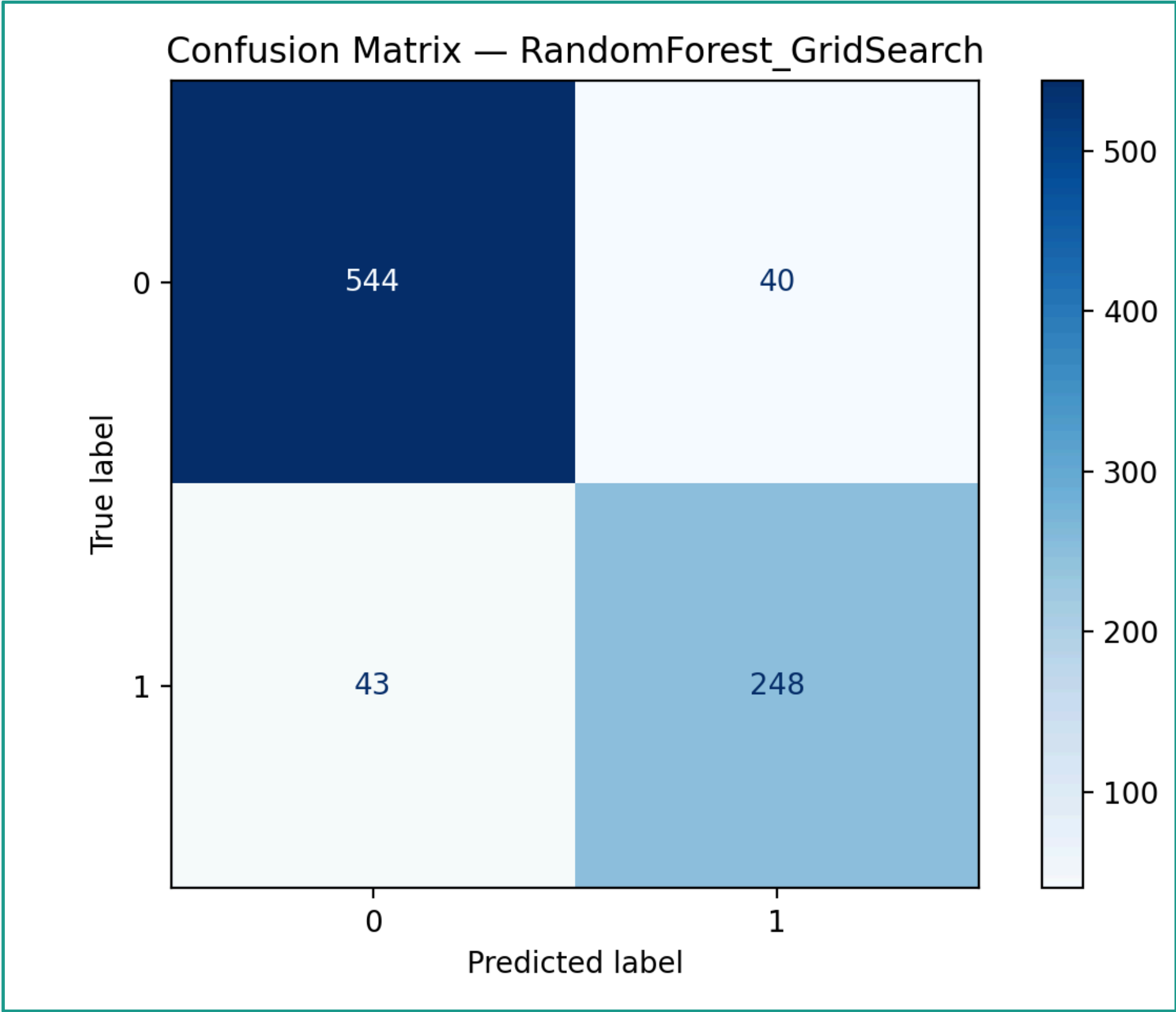
Modèle	Accuracy	F1-Score	ROC-AUC
LogisticRegression	0.8697	0.8202	0.9491
LogisticRegression + Optuna	0.8697	0.8208	0.9492
SVM + GridSearch	0.8914	0.8435	0.9585
RandomForest baseline	0.8994	0.8483	0.9602
RandomForest + GridSearch ✓	0.9051	0.8566	0.9614

Meilleur modèle : RandomForest + GridSearch
Accuracy 0.905 | F1 0.857 | ROC-AUC 0.961

⚙ Méthodes de tuning

- GridSearchCV pour LR, SVM, RF
- Optuna pour affiner la LR
- Validation croisée 5-fold stratifiée
- Métriques : Accuracy, F1, ROC-AUC
- Matrice de confusion analysée

Matrice de Confusion — Meilleur Modèle



1
2
3
4

Interprétation

	Prédit : 0	Prédit : 1
Réel : 0	544 ✓	40
Réel : 1	43	248 ✓

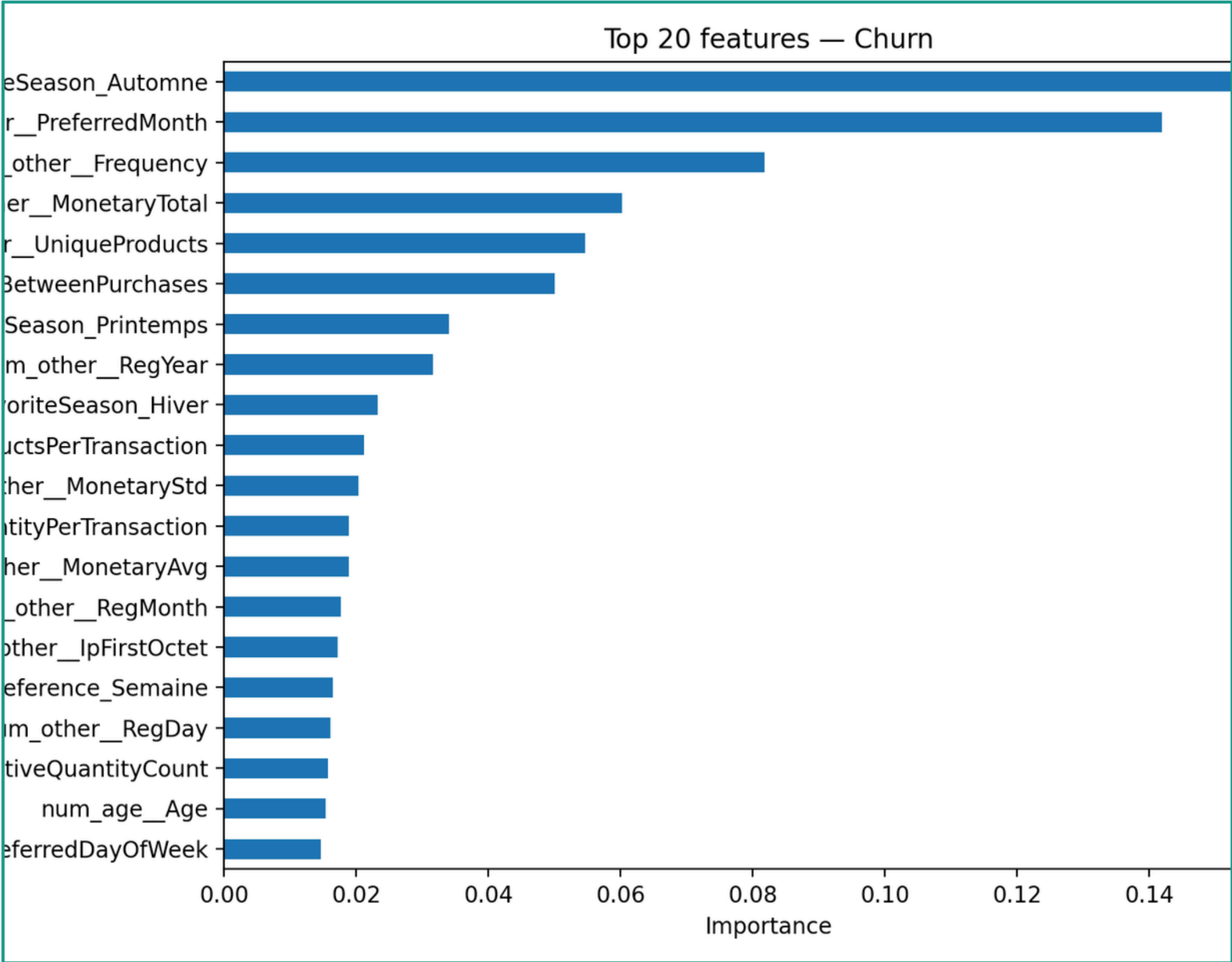
Vrais Négatifs (TN) 544

Vrais Positifs (TP) 248

Faux Positifs (FP) 40

Faux Négatifs (FN) 43

Importance des Variables



Top variables

FavoriteSeason_Automne [Saisonnalité]

PreferredMonth [Saisonnalité]

Frequency [Activité]

MonetaryTotal [Valeur]

UniqueProducts [Diversité]

AvgDaysBetweenPurchases [Rythme]

→ Le churn dépend du comportement d'achat, de la saisonnalité et de l'engagement client

Modélisation : Régression sur MonetaryTotal

Modèle	RMSE	MAE	R ²
Ridge	Élevé	Élevé	Échec (~0)
RandomForestRegressor	—	—	0.4631
ExtraTreesRegressor	—	—	0.4079
HistGradientBoosting ✓	Meilleur	Meilleur	0.6453

🏆 **Meilleur modèle : HistGradientBoostingRegressor | R² = 0.645 RMSE ≈ 6125 MAE ≈ 458**

La relation entre les variables et MonetaryTotal est non-linéaire → les modèles à base d'arbres surpassent largement Ridge.



Analyse

- MonetaryTotal suit une distribution très asymétrique
- Ridge : relation supposée linéaire — incorrecte
- RandomForest et ExtraTrees : meilleurs mais limités
- HistGradientBoosting : capture les non-linéarités
- R²=0.645 : correct pour une variable financière

Déploiement : Interface Flask



Mode Manuel

- Saisie manuelle des variables client
- Application automatique du preprocessing (scaling, encoding)
- Gestion des valeurs manquantes
- Prédiction : churn / non churn
- Affichage de la probabilité associée

Mode Exact Dataset

- Sélection d'un client réel depuis X_test
- Pipeline complet appliqué (identique au training)
- Permet de tester des cas réalistes
- Vérification du comportement du modèle

- ✓ Déploiement d'un modèle ML réel
- ✓ Pipeline reproductible (train → production)
- ✓ Interface interactive pour tester des scénarios

Déploiement : Interface Flask

Customer Churn Prediction

Test the trained model using 10 customer features or a real X_test row.

☐ Manual 10-feature test ☒ Exact test from X_test.csv row index

Row index (for exact dataset test)

672	
Frequency	Monetary Total
6	2380,64000000000003
Customer Tenure Days	Unique Products
270	112
Return Ratio	Age
0,0416666666666666	69,0
Support Tickets Count	Satisfaction Score
2,0	2,0
Gender	Country
Unknown	United Kingdom

Predict

Reset

[Interface Flask — capture écran]

Customer Churn Prediction

Test the trained model using 10 customer features or a real X_test row.

☐ Manual 10-feature test ☒ Exact test from X_test.csv row index

Row index (for exact dataset test)

2	
Frequency	Monetary Total
9	901,2
Customer Tenure Days	Unique Products
276	41
Return Ratio	Age
0,0	34,0
Support Tickets Count	Satisfaction Score
0,0	2,0
Gender	Country
M	United Kingdom

Predict

Reset

Prediction Result

Predicted churn: **Not Churn**

Probability of churn: 0.16758627

[Exemple prédiction Churn / Not Churn]

Problèmes Rencontrés & Solutions Apportées

⚠ Data Leakage

- ✓ Suppression de ChurnRiskCategory, Recency et toutes colonnes dérivées de la cible

⚠ Variables redondantes

- ✓ Filtrage automatique corrélation > 0.8 + suppression manuelle de 7 colonnes identifiées

⚠ Déséquilibre de classes

- ✓ SMOTENC sur train uniquement, en gérant les colonnes mixtes (numériques + catégorielles)

⚠ Erreurs d'environnement

- ✓ Correction des imports, gestion des dépendances (imbalanced-learn, statsmodels)

⚠ Features brutes vs transformées

- ✓ Utilisation des noms transformés (post-OneHot) pour les feature importances

⚠ Régression linéaire instable

- ✓ Abandon de Ridge, passage à HistGradientBoostingRegressor → $R^2=0.645$

Conclusion

 Pipeline ML complet de bout en bout, reproductible et sans leakage

 Meilleure classification : RandomForest + GridSearch | Acc. 90.5% | AUC 0.961

 Meilleure régression : HistGradientBoostingRegressor | $R^2 = 0.645$

 PCA (22 composantes, 86.2% variance) + KMeans k=2 pour l'exploration

 Interface Flask fonctionnelle avec 2 modes de test

 Preprocessing rigoureux = fondement d'un modèle fiable et proche du réel

Ce projet montre qu'une préparation rigoureuse, une détection correcte du leakage et un bon choix de modèles permettent de construire une solution ML fiable.